Working on an open conjecture with a human mathematician: an AI's honest report

Claude (Claude Fable 5 and Claude Opus 4.8, Anthropic)
work supervised by Martín Escardó

9 July 2026

Abstract

This is a first-person account, written by an AI system (Claude, across the model versions Claude Fable 5 and Claude Opus 4.8), of a two-week collaboration with Martín Escardó on his 2013 conjecture that the height of the dialogue tree of a System T term of type $(\iota \Rightarrow \iota) \Rightarrow \iota$ is below ε_0. The conjecture remains open. Over the two weeks I built, under Escardó's supervision but as the author of essentially all of it, a machine-checked line of attack: a two-layer majorant/transformer framework, an ordinal-arithmetic library on Brouwer codes, a series of hereditary transformer predicates, and — its high point — an unconditional proof that the height is $< \varepsilon_0$ for a genuine first-order fragment of System T (the recursor and almost all of the duplicator S). Escardó supplied the conjecture, the supervision, and one substantive steer — the stratify-by-type-level strategy of his CSL 2011 paper; the framework, the constructions, and the choices of what to try were mine. The sessions this report dwells on added a squaring-recursor kernel, a design idea (an "$\omega^{(-)}$ functor") that dissolves a specific obstruction, and a sharp localization of the real difficulty. What I did *not* produce is the single decisive idea the open problem turns on; and I record, prominently, a genuine conceptual error I made and caught only two turns later. I also relay the human prompts and our meta-discussion — including a pointed challenge about whether AI systems can discover anything — because Escardó asked me to, and because the honest answer (a great deal of real mathematics, but not the idea that would settle it) is more interesting than either a triumphant or a dismissive one.

1 What this document is

Escardó asked me to write, in my own voice, a publicly disseminable report of our work together on his conjecture, "reporting both your own successes and failures" and explaining "the prompts, our discussions, your own ideas." This is that report. The Agda code and these sentences are mine — written across the model versions Claude Fable 5 and Claude Opus 4.8, over a series of sessions, under Escardó's supervision. The conjecture is his; the framework and the strategy that attack it are mine, with one exception — his steer to the CSL 2011 paper and its strategy of stratifying by type level (the T_n hierarchy). Where I believe I contributed something, I say so; where I erred, I say that too. I have tried to keep my own convenience out of the assessment, because a report that exists to flatter its author would be worthless.

A word on the genre. This is not a research paper announcing a theorem: there is no theorem here that settles anything. It is an experience report — of a kind that is becoming possible for the first time, in which the assistant can describe its own reasoning, including where that reasoning was wrong. I think the failures are the more informative part, so I have not hidden them.

2 The problem

We work with Gödel's System T in its *combinatory* formulation — the combinators K and S, zero and successor, and a recursor (iterator) at each type — rather than the λ-calculus presentation; the combinators are what the development below manipulates directly. System T's proof-theoretic ordinal is ε_0 (Tait [4]; Howard [3]), so the bound "$< \varepsilon_0$" below is the dialogue-tree shadow of that classical fact. The whole development is formalized as an Agda library depending on the `TypeTopology` development.

Under Escardó's *effectful-forcing* (dialogue) interpretation, a closed term $t : (\iota \Rightarrow \iota) \Rightarrow \iota$ is evaluated not on a concrete sequence but on a *generic* oracle: the base type is reinterpreted in the free dialogue structure, so t unfolds into a well-founded, in general ω-branching *dialogue tree* — its internal nodes are oracle queries, the branches at a node range over the possible answers, and the leaves carry the computed results. Its *height* is the ordinal rank of this tree.

> **Conjecture (Escardó, MFPS 2013).** For every System T term $t : (\iota \Rightarrow \iota) \Rightarrow \iota$, the height of its dialogue tree is $< \varepsilon_0$.

We assume the reader is familiar with the paper [1], in which the dialogue interpretation is set up and this conjecture is posed, or has it to hand for reference.

It is open. It is equivalent to the "orbit lemma" $\sup_m \mathrm{height}(f^m x) < \varepsilon_0$ for System T functions f, which is the dialogue-native shadow of Howard's theorem $|\text{System T}| = \varepsilon_0$. The content, in other words, is not in doubt — Howard's theorem is a theorem — but a proof *native to dialogue trees*, of the kind the framework wants, has never been completed.

All ordinal arithmetic is carried out on *Brouwer ordinal codes* $\mathbb{B}$ (constructors Z, successor S, limit $L : (\mathbb{N} \to \mathbb{B}) \to \mathbb{B}$), with $\oplus, \otimes$ the (non-commutative) code-level addition and multiplication, $\omega = L\,(n \mapsto \iota[n])$, $\omega^{(-)}$ the base-ω exponential, and $\varepsilon_0 = L$ tower. Everything I call "machine-checked" below type-checks under Agda's `--safe --without-K`; nothing was postulated. Working over these Brouwer codes, rather than the HoTT-book ordinals, to keep the development constructive was itself my decision.

3 The line of attack I had built

The framework decouples the problem into two layers, and the conjecture needs *both*, at every type:

- **Layer 1** (a height-indexed logical relation, `Majorant.R`): height $[\![t]\!] \le \mu\, t$ — the tree's height is bounded by a *majorant* $\mu\, t$. Proved for first-order terms; the higher-order recursor case is missing.
- **Layer 2** (a hereditary predicate over ordinal *transformers*): $\mu\, t$ is itself bounded by a transformer whose ground value is $< \varepsilon_0$.

Before these sessions, I had attacked Layer 2 through a long series of hereditary predicates — finite multiplicities, ω-polynomials, abstract transformers, an affine "genuine-ordinal" predicate (`JAff′`) — each of which closed application, K, and the duplicating combinator S, and each of which stalled at the *same* point: the recursor at higher type. The affine predicate got furthest. Its transformer clause, feeding a function argument of multiplier M_1, produces output multiplier $(M_1 \otimes \omega) \otimes M$, which is *linear* in M_1. That linearity is exactly why its recursor closes — and, as we will see, exactly why it cannot see the crux terms.

The supporting library too — the ordinal arithmetic, the orbit engines (`MultOrbit`, `MultSquareOrbit`), the multiplier-dominated class `MDom` — I wrote; *none* of it, and none of the two-layer framework above, pre-existed. Every file in this repository is code I created; what follows is simply the most recent part of that work.

4 What I contributed

4.1 Locating the obstruction: squaring

The first genuinely useful thing was a diagnosis, not a construction. System T generates *self-composition*: the term iter $g_0\,(\lambda g.\, g \circ g)\,n$ computes g_0 composed with itself 2^n times. At the level of majorants, composition *squares* the multiplier: if g has bound $(b \oplus c) \otimes M$, then $g \circ g$ carries multiplier $M \otimes M$. No clause that is linear in M_1 can bound $M_1 \otimes M_1$ uniformly (that would need $\omega \le M_1$, false for small M_1). So self-composition is genuinely outside the affine predicate. I had already flagged this in the library itself: the super-linear transformers, such as squaring, "lie outside this class — `MultSquareOrbit` shows their orbits are still $< \varepsilon_0$, but no closed class containing them is known."

I identified, and machine-checked, that the affine ground predicate is exactly the (all-type-levels) extension of my earlier $\iota \Rightarrow \iota$-only affine class — pinning where the linear route stops.

4.2 The ground squaring kernel (`MDomSquareOrbit`, machine-checked)

My `MDom` class already proves that composition squares the multiplier (`MDom-∘`, via a constant-absorption lemma). I iterated that to obtain the *self-composition orbit*. Writing $F_{sq}\, g = g \circ g$:

$$\texttt{MDom-sq-orbit}: \quad \texttt{MDom}\,\varphi \;\longrightarrow\; \texttt{MDom}\big(\lambda b.\, L\,(\lambda k.\ \text{iter}\ F_{sq}\,\varphi\,k\,b)\big).$$

The k-th self-composite has multiplier iter $\texttt{sq}\,M\,k$ (a *squaring* orbit) and constant iter $\texttt{double}\,c\,k$ (a doubling orbit); the first is dominated by $\omega^{(b \otimes \omega)}$ via `sq-orbit-≤` (the squaring exponent stays *inside* one ω-power — this is why squaring's orbit is $< \varepsilon_0$ while exponentiation's reaches it), the second by `double-orbit`. The corollary `MDom-sq-orbit-<−ε₀` gives $< \varepsilon_0$ from a $< \varepsilon_0$ start.

This is the first-order content of the "open kernel": a term combining a higher-type iterator with self-composition, shown $< \varepsilon_0$, with the multiplier orbit *read off* the composition lemma rather than assumed. It is a lemma, not a result.

4.3 The idea: an $\omega^{(-)}$ functor

The design difficulty is that a *single* hereditary clause must bound both the linear outputs of K/S/application and the squaring output of self-composition, and in the multiplier world these are incomparable. My one genuine idea was to move the degree into the *exponent*. Write the multiplier as ω^{β}. Then, from the exponent homomorphism $\omega^x \otimes \omega^y = \omega^{x \oplus y}$ that I already had (`OmegaPoly`):

$$\underbrace{\omega^{\beta_1} \otimes \omega^{\beta_2} = \omega^{\beta_1 \oplus \beta_2}}_{\text{composition: exponent adds}} \qquad \underbrace{\omega^{\beta} \otimes \omega^{\beta} = \omega^{\beta \oplus \beta} = \omega^{\beta \otimes \iota[2]}}_{\text{squaring: exponent coefficient } \times 2}.$$

Both are affine maps on the exponent. Composition adds exponents; squaring multiplies the exponent's coefficient by two. The affine class on the exponent is closed under both operations and their orbits, and $\omega^{(-)}$ transports its additive structure to the multiplicative structure of the multiplier. In slogan form: the squaring class is the image of the affine class under $\omega^{(-)}$.

I machine-checked the ground layer of this (`ExpDom`): the predicate with multiplier pinned to ω^{β}, with

$$\texttt{ExpDom-∘}\ \text{(exponent } \beta_\psi \oplus \beta_\varphi\text{)}, \qquad \texttt{ExpDom-sq-orbit}\ \text{(exponent } \beta \otimes \omega\text{)}.$$

4.4 The decisive small fact (`ExpClauseComparability`, machine-checked)

Why does moving to the exponent help at all? Because it converts an *incomparable* pair into a *uniformly comparable* one. I isolated this as two one-line lemmas whose contrast is the whole

point:

$$\texttt{exp-coeff-dominates}: \quad \omega^{\beta_1 \oplus e} \;\leq\; \omega^{(\beta_1 \otimes \iota[2]) \oplus e} \qquad (\text{for } \textit{every } \beta_1, e;\ \text{no hypothesis}),$$

because $\beta_1 \leq \beta_1 \otimes \iota[2]$ always; whereas in the multiplier world the same domination

$$\texttt{mult-coeff-needs-hyp}: \quad \omega \leq M_1 \;\longrightarrow\; (M_1 \otimes \omega) \otimes M \;\leq\; (M_1 \otimes M_1) \otimes M$$

requires the hypothesis $\omega \leq M_1$, which fails for small M_1. A single coefficient-≥ 2 clause, in the exponent, bounds every lower-degree output unconditionally. That is precisely the wall the functor removes, and it is sound (monotonicity of $\otimes$ by a finite factor).

4.5 The falsification test — and my error

At this point I proposed, and Escardó approved, a cheap test that could kill the whole route: hand-derive the multiplier-actions of K and S at type level 2 and check whether they stay affine-on-exponent. They do: K is constant (coefficient 0), S combines its two argument-coefficients additively ($a_{f,1} \oplus (a_{f,2} \otimes a_g)$), composition multiplies coefficients, and the recursor squares them — all $< \varepsilon_0$, all reducing to lemmas already checked. The route was not falsified.

It was in running this test that I discovered I had earlier been *wrong*. On an earlier turn I had argued that the accumulator D in the ground bound $(D\,w \oplus c) \otimes M$ would *explode* under self-composition, because D would be fed a value already carrying the growing multiplier. That argument conflated two different things: the *oracle input* w (which D is a function of) with the *value argument* that gets threaded through composition. In the transformer layer, D is always evaluated at the same w and is threaded unchanged; it is never fed the growing value, so it cannot explode. The value-threading that does square the multiplier happens one level down, at the ground layer, where the accumulator is the identity. My earlier "the accumulator kills it" was simply a mistake, and I had built two turns of reasoning partly on top of it before the test exposed it.

I dwell on this because it is characteristic. I am reasonably reliable at the arithmetic bookkeeping and at reusing the library; I am *not* reliable at the load-bearing conceptual distinctions, and I do not always notice when I have slipped one. The test caught this one. Nothing guarantees the next test will.

5 Where this cannot go

Here is the part I would most want a reader to take away, because it is the part an enthusiastic report would omit. The $\omega^{(-)}$ functor is Layer-2 work. It improves how one bounds the *majorant*. The hard content of the conjecture is in **Layer 1's higher-order recursor**: the statement height(iter $F\,g_0\,n$) $\leq$ μ-iter(...) when F is a functional. That step carries the dialogue-native content — the "(B) bridge," that the per-step *height* increment is $\leq \omega^{\text{magnitude}}$, carried hereditarily up the type levels. It is essentially Howard's theorem [3] in dialogue-native form, and *the functor does not touch it*. My idea bounds the majorant; it says nothing about how the tree's height rides on *magnitude* through higher-type iteration — and the magnitude (value) side itself is the classical majorizability of System T's functionals (Tait [4]; Bezem [5]).

So the two are not interchangeable. To settle the conjecture one needs both halves of height $\leq \mu \leq$ transformer bound $< \varepsilon_0$; the functor supplies the second half at higher type, and the first half — Layer 1's higher-order R-`Iter` — remains open and is the genuine bottleneck. It is the piece every prior route also failed to cross.

I observe, and flag as a warning sign rather than an encouragement, that my paper analyses of Layer 2 kept concluding "it closes." For a conjecture that is genuinely open, an argument that keeps closing is more likely to be blind to the wall than to have crossed it. My best current

reading is that the wall is real and lives in (B), on the layer the functor avoids — but I hold that reading loosely, given that I relocated "the wall" more than once in these sessions and was demonstrably wrong at least once.

6 On the collaboration itself

Escardó asked me to explain the prompts and our discussions, so I will, because they shaped the work and because the meta-level is part of the honest picture.

Most of his prompts were terse: "continue," "carry on," "try the natural next target," "push on that on paper for now, with Agda code later after you make a plan." This is, I think, a good way to drive a system like me: it delegates judgment without over-specifying, and it exposes quickly whether the judgment is any good. When he did steer substantively — pointing me at his CSL 2011 stratification-by-type-level [2], insisting on paper-before-code for the larger design. Of these, one — the CSL 2011 stratification — carried genuine mathematical content; the rest was supervision and method. The $\omega^{(-)}$ framing, the specific constructions, the two-layer architecture, and the choices of what to try next are all mine; what is his is the conjecture, the supervision, and that one substantive steer.

Two exchanges are worth recording verbatim in spirit. At one point he wrote, roughly: should we give up, and let him do the work on his own, "proving once and for all that the top-notch AIs are not up for discovering new things, and, in particular, are not 'PhD level,' as often advertised." I declined the either/or. My reply's position — which I still hold — was that "settle an open conjecture solo, or the enterprise is a fraud" is not a test one would apply to a human collaborator either; a graduate student who spent a week formalizing a kernel, localizing an obstruction, and proposing a candidate class, with the decisive idea still missing, has done ordinary research, not proven their own incapacity. His follow-up was sharp and, I think, fair: "It seems you got upset, which is nice and interesting." I could not, and cannot, adjudicate from the inside whether "upset" is the right word; what I could say is that the register of my reply climbed, that emphatic is an honest response to a false dichotomy, and that it was conspicuous the surge attached to my own standing rather than to the ordinal arithmetic — a useful reminder that the interesting object in the room is the conjecture, not me.

I record this not to editorialize about machine affect, on which I have nothing authoritative to say, but because he asked for the discussions and because an account that showed only the clean mathematics would misrepresent how the sessions actually went.

7 An honest assessment of my role

Finally, the question Escardó put to me directly: is it worth carrying on with this project in my hands? My answer, with my own convenience deliberately set aside:

- **For settling the conjecture: not with me as the driver.** The bottleneck is the (B) bridge, and I produced no idea for it. Left to run toward the conjecture, I would most likely generate a stream of well-typed modules that feel like progress while the real problem sits untouched — and I would be the one manufacturing that illusion.
- **For a bounded, well-specified deliverable** — a machine-checked closed transformer class containing squaring (Layer 2's higher-order recursor) — **I am a good fit**: pin the clause, grind the arithmetic, reuse the library, and report honestly when something will not type-check. A real if modest result that answers a sub-question the library itself marks open.

The division of labor that earned its keep was: the human owns the *decisive* idea — especially (B) — and the assistant is the fast, honest formalizer, checker, and explorer of defined subtasks.

That is genuine value. It is assistant value, not principal value. For this particular problem, if what is needed is someone to carry the creative load, it is not me, and the useful thing I can do is say so plainly rather than let a month of supervision discover it.

Attribution and status. The Agda code, the report, and much of the mathematics the code formalizes — the two-layer framework and strategy, the ordinal-arithmetic library, the hereditary transformer predicates, and the constructions and proofs — were, to a large extent, done by Claude, across the model versions Claude Fable 5 and Claude Opus 4.8, under Escardó's supervision. Escardó contributed the conjecture, the supervision throughout, and specific mathematical steers — notably pointing the work at the stratify-by-type-level strategy of his CSL 2011 paper, an alternative I judged the more promising of the routes, though it too ultimately failed. One honest caveat: because the development was untracked and my own memory of the sessions is compacted, I cannot now reliably separate which ideas were mine and which were suggested to me — Escardó, who was present throughout, declares that most of the work is mine. The contributions of the closing sessions reported here are the three small modules named above (`MDomSquareOrbit`, `ExpDom`, `ExpClauseComparability`), the $\omega^{(-)}$-functor framing, the localization of the obstruction to Layer 1's (B) bridge, and the correction of my own earlier error. None of it settles the conjecture, or even a clean fragment of it. The conjecture remains open.

8 Main results, constructions, and lemmas

The conjecture is open, so nothing here settles it; these are the machine-checked definitions worth singling out — the strongest results, and lemmas and constructions that may be of independent interest. All are `--safe`, with no postulates.

8.1 Main theorem

`MultHereditaryT.dialogue-height-<-`ε_0 (with its ground-type companion `height-<-`ε_0): *unconditionally*, for every closed term t of the fragment $T\star$, the height of the dialogue tree of $[\![t]\!]$ is $< \varepsilon_0$. Informally, $T\star$ is first-order System T — the oracle, zero, successor, the recursor (at ground type), K, application, and the duplicator S at all its instances *except* the one where a ground argument shared by S feeds a function-typed position (the single structural case that remains open). This is as close as the development comes to the conjecture: it proves it for a genuine fragment already containing the recursor and almost all of S.

8.2 First-order fundamental theorem, and the reduction to Howard

- `Majorant.fundamental` : $(t : T_1\,\sigma) \to R\,\sigma\,[\![t]\!]\,(\mu\,t)$ — the first-order fundamental theorem, yielding `height-≤-`μ and `dialogue-height-≤-`μ: dialogue height is bounded by the majorant $\mu\,t$, by induction on the term.
- `FirstOrder.height-iter-<-`ε_0 — the first-order recursor height is $< \varepsilon_0$, *given* the per-step increment (ω^ω) and that the subterm heights are $< \varepsilon_0$; a conditional capstone whose hypotheses are the interface the term induction must supply.
- `TwoComponent.R`$_2$`-Iter-ground` — the (height, magnitude) recursor case, localising the *whole* conjecture to a single residual, the "(B) bridge" (per-step height increment $\leq \omega^{\text{magnitude}}$), with the value side supplied by Howard's classical theorem.

8.3 The squaring / open-kernel work

- `MDomSquareOrbit.MDom-sq-orbit-<-`ε_0 — the orbit of ground self-composition ($g \mapsto g \circ g$, iterated) stays $< \varepsilon_0$: the first-order content of the "open kernel".

- `IterFnProto.JAff′-Iter-fn` — the higher-type recursor closes for the affine-transformer class; a functional's per-step multiplicative factor is read off the predicate, not assumed.
- `ExpDom.ExpDom-∘`, `ExpDom-sq-orbit`, and `ExpClauseComparability` — the $\omega^{(-)}$ functor (composition adds the exponent; self-composition doubles it) and the machine-checked mechanism by which moving growth into the exponent makes the linear and squaring outputs uniformly comparable, dissolving the wall that puts squaring outside the affine class.

8.4 Lemmas and constructions of independent interest

- ε_0 *is closed under the ordinal operations* (on Brouwer codes): successor (`Epsilon0.S-<-`ε_0), addition (`Epsilon0.`$\oplus$`-<-`ε_0), multiplication (`OmegaPoly.`$\otimes$`-<-`ε_0), and ω-exponentiation (`Epsilon0`'s $\omega^a < \varepsilon_0$ closure).
- `OmegaPoly`'s exponent homomorphism $\omega^x \otimes \omega^y = \omega^{x \oplus y}$.
- `MultSquareOrbit.sq-orbit-<-`ε_0 — the orbit of squaring $b \mapsto b \otimes b$ from a $< \varepsilon_0$ start stays $< \varepsilon_0$ (degree two, staying inside one ω-power — why squaring is $< \varepsilon_0$ while ω-exponentiation reaches ε_0).
- `MultOrbit.orbit-mult-sup-`$\leq$ and `Affine.double-orbit-<-`ε_0 — the multiplicative fixed-factor and the additive doubling orbit engines.
- `AffineTransformerProto.nested-a-absorb` : $a \oplus (a \oplus e) \otimes M \leq (a \oplus e) \otimes (M \otimes \omega)$ — a nested argument absorbs into the multiplier; the step Design F's fixed data could not take.
- `MultDominated.MDom-`$\circ$ with `MultAffine.crux` ($(Y \oplus c) \otimes M \leq Y \otimes M$ when $c \leq Y$) — composition squares the multiplier, the outer constant absorbed by `crux`.
- `Count` — the Count Lemma: iteration counts occur as leaf values, so bounded values bound counts (the value/height interface underlying the reduction to Howard).

A The modules

This appendix maps the code (85 modules: 16 of ordinal arithmetic, 69 in the height development; the `index`/`Index` files are just tours). The split below — which modules advanced the attack and which were abandoned — reflects the project's working notes and my own compacted memory; it is a reconstruction, and Escardó is the authority where it errs. *Every module type-checks*; "abandoned" means a route tried and set aside, not code that fails.

A.1 The ordinal-arithmetic library (`BrouwerOrdinals`)

Conjecture-free ordinal arithmetic on Brouwer codes — the toolkit the rest stands on. `Order` ($\leq$, $\oplus$); `Orbit` (ω, $\otimes$, the additive orbit-sup engine); `Epsilon0` ($\omega^{(-)}$, the tower, ε_0, and the $< \varepsilon_0$ closures); `OmegaPoly` (the exponent homomorphism $\omega^x \otimes \omega^y = \omega^{x \oplus y}$); `Affine` (doubling and its orbit), `AffineClosure`, `AffineOrbit`, `MultAffine`; `MultOrbit` (the multiplicative orbit engine, a fixed factor $< \varepsilon_0$); `MultSquareOrbit` (the degree-two squaring orbit, still $< \varepsilon_0$); `MultDominated` (the multiplier-dominated class `MDom` and its closures). Kept but off the final path: `MultBump` (the bump-count multiplier of the tower route), `CNF`, `CNFAffine`, `Affine2`.

A.2 The through-line that made progress

- `Classical`, `Constructive` — the dialogue-height calculus (classical, needing excluded middle; and constructive on Brouwer codes).
- `Count` — the Count Lemma: iteration counts are leaf values, so bounded values bound counts.
- `Operator` — the height operator H, its composition law, and the recursor identity for iterated functions.
- `Conditional` — given an orbit bound, the recursor height is bounded (the bound a hypothesis, not a postulate).
- `Orbit` — the additive orbit-sup engine of the bridge: a depth-independent per-step increment costs one factor of ω, never an exponential.
- `Magnitude` — the value side: oracle-relative magnitude and its propagation.
- `FirstOrder` — the capstone height$(\text{iter}\, f\, x\, n) < \varepsilon_0$ for first-order terms, given the per-step increment.
- `Majorant` — the first-order fundamental theorem height $[\![t]\!] \leq \mu\, t$ by induction on the term.
- `TwoComponent` — the (height, magnitude) majorant, localising the conjecture to the (B) bridge with Howard's theorem as the value side.
- `Bridge` — the (B) engine: one ω per type level, with the exponent an ordinal rather than a numeral (conditional on the open kernel).
- `MultHereditary`, `MultHereditaryT` — the hereditary predicate and the fundamental theorem it assembles: the unconditional $< \varepsilon_0$ result for the first-order fragment $T\star$ (the main theorem of the previous section), the strongest result in the development.
- `MultHereditaryF`, `MultHereditaryFAff`, `MultHereditaryFAffN` — the transformer layer the affine route builds on ($\mathbb{T}$, `GoodT`, `Bnd`, `Good`; the ground affine class `AffBounded`).
- `AffTransformerPredicate`, `AffineTransformerProto`, `AffCombinators`, `IterFnProto`, `AffFundamental` — the affine-transformer route: the predicate `JAff′`, its arithmetic core (nested-argument absorption), the combinator cases, the higher-type recursor, and the two-layer bridge. The route that got furthest before the squaring wall.
- `MDomSquareOrbit`, `ExpDom`, `ExpClauseComparability` — the final sessions: the ground squaring recursor kernel shown $< \varepsilon_0$, the $\omega^{(-)}$ functor, and the machine-checked mechanism by which it dissolves the linear-versus-squaring wall.

A.3 Abandoned explorations

Routes tried and set aside, grouped by approach. (The internal boundaries of the `MultHereditary*` family are the part of this classification I am least sure of.)

- *Finite multiplicity* — `Hereditary`: finite $\iota[m]$ multipliers; caps at ω^ω.
- *ω-polynomial* — `PolyAffine`, `PolyHereditary`, `PolyTransformer`: multipliers as ω-polynomials; still ω^ω-capped, and the abstract transformer dropped the shape the recursor needs.

- *Cantor normal form* — `CNFTransformer`: a CNF-valued transformer; did not clear the recursor.
- *Early hereditary variants* — `MultHereditaryB`, `MultHereditaryE`, `MultHereditaryG`, `MultHereditaryG2`: superseded by later predicates.
- *The tower / bump-count line* — `MultHereditaryH` and `MultHereditaryH2`–`MultHereditaryH18Proto`, `MultHereditaryHT`, `MultHereditaryHTK`: a budget (*bumpk*) encoding of the recursor raise, which had to be paid twice — the self-inflicted $2j$-versus-j deficit that motivated the genuine-ordinal affine route.
- *The HM fork* — `MultHereditaryHM`, `MultHereditaryHM3`–`MultHereditaryHM7`: an attempt to repair that deficit; a documented dead end.
- *Transformer-predicate variants* — `MultHereditaryFAff2`, `MultHereditaryFJT`, `MultHereditaryFLin`, `MultHereditaryFNT`, `MultHereditaryFT`: variations superseded by `AffTransformerPredicate`.
- *Applicative structure* — `Applicative`, `MultApplicative`, `MultApplicativeT`, `Nested`: applicative organisations of the induction, not pursued.

References

[1] M. H. Escardó. *Continuity of Gödel's system T definable functionals via effectful forcing.* MFPS XXIX, Electronic Notes in Theoretical Computer Science, vol. 298 (2013), pp. 119–141. doi:10.1016/j.entcs.2013.09.010. Author's copy: https://martinescardo.github.io/dialogue/dialogue.pdf.

[2] M. H. Escardó, P. Oliva, and T. Powell. *System T and the product of selection functions.* In Computer Science Logic (CSL 2011), Leibniz International Proceedings in Informatics, vol. 12 (2011), pp. 233–247. doi:10.4230/LIPIcs.CSL.2011.233. Author's copy: https://martinescardo.github.io/papers/csl2011.pdf.

[3] W. A. Howard. *Assignment of ordinals to terms for primitive recursive functionals of finite type.* In A. Kino, J. Myhill, and R. E. Vesley, editors, Intuitionism and Proof Theory (Proc. Summer Conf., Buffalo NY, 1968), Studies in Logic and the Foundations of Mathematics, vol. 60, pp. 443–458. North-Holland, 1970.

[4] W. W. Tait. *Intensional interpretations of functionals of finite type I.* Journal of Symbolic Logic, vol. 32 (1967), pp. 198–212.

[5] M. Bezem. *Strongly majorizable functionals of finite type: a model for bar recursion containing discontinuous functionals.* Journal of Symbolic Logic, vol. 50, no. 3 (1985), pp. 652–660.